MONK.COM · SYSTEM DESIGN EXPLAINED

How Monk Works Under the Hood

A plain-English guide to every service, diagram, and technical term inside Monk's Accounts Receivable automation platform — built for founders, CFOs, and finance teams who want to understand what their engineering team built.

CONTENTS

01 · **The Big Picture** — How all 5 services connect, from contract to cash

02 · **Contract Service** — PDF upload → AI extraction → billing schedule

03 · **Invoice Service** — Scheduling, generating and emailing invoices

04 · **Payment Service** — Collecting money securely via Stripe

05 · **Collections Agent** — AI-powered follow-ups with 24% higher response rate

06 · **Reporting Service** — DSO, aging buckets, real-time cash view

07 · **Glossary Part 1** — Webhooks, TLS, Message Queue, JWT, Redis…

08 · **Glossary Part 2** — ACID, HMAC, ERP, LLM, Multi-tenancy, Cron…

01 The Big Picture

Monk is made of 5 specialised services that hand work off to each other — like departments in a company. Here is how a contract becomes cash in your bank account.

OVERVIEW · ALL SERVICES

THE JOURNEY: CONTRACT → CASH

① Contract Service
Reads your PDF. Extracts billing terms using AI.

→

② Invoice Service
Generates invoices on the right dates. Sends to customers.

→

③ Payment Service
Accepts card payments via Stripe. Records money received.

→

④ Collections Agent
If unpaid, AI sends personalised follow-up emails.

→

⑤ Reporting Service
Tracks everything. Shows DSO, aging, and cash collected.

INFRASTRUCTURE THAT HOLDS EVERYTHING TOGETHER

🔒 API Gateway — The Front Door

Every request from the Finance dashboard, customer payment portal, and external systems enters here. Checks who you are (Auth), limits traffic (Rate Limiting), and decrypts secure connections (TLS Termination).

↓

📬 Message Queue — The Shared Inbox

Services leave messages for each other here instead of calling each other directly. When a payment is recorded, the Payment Service posts "payment.recorded" — the Reporting Service and Collections Agent both see it and react. One service failing cannot crash the others.

↓

🗄️ Primary Database (PostgreSQL)
Stores all invoices, payments, contracts. The single source of truth. Every financial operation is written here first.

⚡ Cache (Redis)
Super-fast memory store. Holds frequently-read data like invoice tokens and dashboard snapshots so the database isn't hit every request.

📊 Analytics DB (ClickHouse)
Stores historical trends for lightning-fast reporting across millions of invoice rows — DSO over 12 months, aging trends.

👥 WHO USES MONK

Finance teams & CFOs upload contracts and check the dashboard. **Customers/payers** receive invoices and click a payment link — no account or login needed. **ERPs like NetSuite** sync data automatically via webhooks.

💡 WHY IT'S BUILT THIS WAY

Each service is **independent** — if the AI collections agent goes down, invoices still get sent and payments still get collected. Services only communicate through the Message Queue, so one problem never causes a system-wide crash.

🏦 WHAT THIS MEANS FOR MONK CUSTOMERS

When a Finance team at Profound uploads a contract, Monk's Contract Service reads it, the Invoice Service schedules billing, the Payment Service collects money when the customer pays, and the Collections Agent follows up if a customer is late — all without anyone doing anything manually. The Reporting Service shows the CFO exactly where cash stands in real time.

02 Contract Service

Monk's starting point. A finance team uploads a PDF contract — this service reads it with AI, extracts every billing term, and creates a schedule of future invoices automatically. No manual data entry.

PDF UPLOAD → AI EXTRACTION → BILLING SCHEDULE

STEP-BY-STEP: HOW A CONTRACT BECOMES A BILLING SCHEDULE

① Finance Team Uploads PDF
Drags a signed contract into Monk. File goes to secure cloud storage (S3). Monk immediately returns "Processing…" — no waiting at the screen.

→

② AI Reads the Contract
Monk sends the PDF text to an AI model (OpenAI / Anthropic). AI extracts: amount, schedule, start/end dates, currency, and billing type.

→

③ Validation Check
AI output is never trusted blindly. Monk checks for valid amounts, correct dates, logical structure. Low-confidence? Sent to human review.

→

④ Billing Events Created
Monk creates one database record per future invoice. A 12-month contract = 12 billing events, each with a scheduled date and amount.

→

⑤ System Notified
"contract.parsed" posted to the Message Queue. Invoice Service picks this up and starts watching those scheduled dates.

THREE TYPES OF BILLING MONK HANDLES

🔁 **RECURRING**

Same amount, same day every period. Example: **$10,000 every month for 12 months.** Monk pre-creates all 12 billing events at upload time. Amount is known upfront.

🏁 **MILESTONE**

Payment triggered by an event, not a date. Example: **$50,000 on project go-live.** Monk waits for the customer to approve the milestone, then triggers the invoice.

📊 **USAGE-BASED**

Amount depends on how much was used. Example: **$0.05 per API call.** Monk waits until end of period, receives usage data, then calculates and sends the invoice.

📄 **MONK REAL-WORLD EXAMPLE**

ElevenLabs signs a new enterprise customer. The contract says: $20,000/month for 6 months, invoiced on the 1st of each month, net-30 payment terms. The finance team uploads the PDF to Monk. Within minutes, Monk has read the contract, created 6 billing events (one per month), and scheduled the first invoice for the 1st of next month — without anyone typing a single number.

⚠️ **CONTRACT AMENDMENTS**

If a deal changes mid-way (new price or term), Monk **cancels all future billing events** from the old contract and creates new ones from the amendment date forward. Past invoices are never touched — the audit trail stays clean and accurate.

🔒 **NO DOUBLE BILLING — EVER**

Each billing event has a unique ID based on the contract + period. If the system runs twice (after a crash or retry), the second run sees the event already exists and does nothing. **You can never be invoiced twice for the same period.**

03 Invoice Service

Runs a check every minute for billing events that are due. When it's time, it generates the invoice PDF, emails it to the customer with a payment link, and uploads to any required procurement portals — no human needed.

SCHEDULED BILLING → PDF → EMAIL → PAYMENT LINK

HOW THE INVOICE SERVICE WORKS EVERY MINUTE

⏰ Every 60 Seconds
A background job (cron) wakes up and asks the database: "Which billing events are due right now?"

→

📋 Find Due Events
Gets up to 500 billing events where status = "ready" AND scheduled date ≤ today. Processes up to 20 in parallel.

→

📄 Generate Invoice PDF
Creates invoice record and PDF in a single database transaction — either both succeed or neither does. No partial states.

→

📧 Email Customer
Sends invoice PDF with a unique payment link. Link contains a signed token (JWT) — no customer login required.

→

📢 Notify System
"invoice.sent" posted to Message Queue. Collections Agent and Reporting Service both pick this up automatically.

THE CUSTOMER'S PAYMENT EXPERIENCE

📬 Customer Gets Email
Invoice arrives in their inbox with a clear "Pay now" button. No account or password needed.

→

🌐 Opens Payment Page
Clicks the link. A branded payment page loads — showing invoice details and the amount due.

→

💳 Enters Card Details
Card number goes directly to Stripe — never to Monk's servers. Monk is kept outside of PCI card data scope.

→

✅ Payment Confirmed
Stripe charges the card and notifies Monk via webhook. Invoice marked as paid. Receipt sent automatically.

🔑 WHAT IS A JWT PAYMENT TOKEN?

JWT stands for **JSON Web Token** — think of it as a digitally-signed sticky note embedded in the payment URL. It contains the invoice ID and company ID. Monk verifies the signature to ensure no one has tampered with it. If someone edits the URL to change the invoice number, the signature breaks and Monk rejects it. Tokens expire after 30 days.

🏢 ENTERPRISE PORTAL UPLOADS

Some large customers (F500 companies) require invoices uploaded to their own procurement portals: **Coupa, Ariba, or SAP.** Monk's Portal Uploader worker does this automatically — logging into the portal and submitting the invoice so your finance team never has to touch these systems manually.

💡 WHY "ATOMIC TRANSACTIONS" MATTER HERE

Creating the invoice record and marking the billing event as "invoiced" happen together in one database transaction. If the server crashes halfway through, the database rolls everything back. The next time the job runs, it sees the billing event is still "ready" and tries again safely. A customer can never receive an invoice that doesn't exist in the database — and can never be invoiced twice for the same billing event.

04 Payment Service

The most critical service. Handles money — every operation must be verified, atomic, and impossible to duplicate. A bug here means a lost payment or a double charge, so this is where Monk is most careful.

STRIPE · WEBHOOKS · ATOMIC WRITES · IDEMPOTENCY

THE PAYMENT FLOW — HAPPY PATH

① Customer Clicks Pay
Token verified. Monk creates a Stripe "PaymentIntent" — a pre-authorised payment reservation.

→

② Stripe Charges Card
Card details go straight to Stripe. Stripe handles bank authorisation. Monk never sees the card number.

→

③ Stripe Sends Webhook
Stripe calls Monk with "payment_intent.succeeded." Monk checks the HMAC signature — fake calls are rejected immediately.

→

④ 3-Way Atomic Write
In one transaction: record the payment, update invoice balance, update invoice status. All three succeed or none do.

→

⑤ System Notified
"payment.recorded" posted to queue. Reporting updates. Collections Agent cancels follow-up. ERP sync triggered.

THE THREE-WAY ATOMIC WRITE — WHY ALL THREE MUST HAPPEN TOGETHER

WRITE 1 — RECORD THE PAYMENT

Inserts a row into the payments table. The Stripe Intent ID is the **idempotency key** — if Stripe sends the same webhook twice (it does this as a safety measure), the second insert is silently ignored. The payment is recorded exactly once, no matter what.

WRITE 2 — UPDATE INVOICE BALANCE

Adds the payment amount to the invoice's running total. For partial payments, this keeps a cumulative balance. Example: $5,000 invoice, customer pays $3,000 first — balance now shows **$2,000 remaining.** The next follow-up email will reference this exact figure.

WRITE 3 — UPDATE INVOICE STATUS

If paid ≥ balance → status becomes **"paid."** If paid < balance → status becomes **"partial."** This is exactly what the Collections Agent reads before sending any email. Status "paid" means the agent stops and never sends another follow-up.

⚠️ WHAT ABOUT PARTIAL PAYMENTS?

Monk supports customers paying in instalments. If an invoice is for $10,000 and a customer pays $4,000 first, Monk marks it "partial" and the Collections Agent resets its clock to day 0 — sending an appreciative acknowledgement email rather than a demanding dunning notice.

🚨 WHAT IF THE WEBHOOK IS MISSED?

Stripe retries failed webhooks for up to 3 days. Monk also runs a **nightly reconciliation job** that pulls all payments from Stripe's API for the past 24 hours and compares against Monk's records — automatically catching any payment that slipped through the cracks.

🔔 WHAT IS A WEBHOOK? — EXPLAINED SIMPLY

A webhook is a way for one system to automatically tell another system when something happens. Instead of Monk constantly asking Stripe "has the payment gone through yet?" (wasteful), Stripe calls Monk's server the moment a payment succeeds — like a doorbell. Monk verifies it's really Stripe calling (via HMAC signature — a mathematical fingerprint only Stripe can produce), then processes the payment. This is how Monk knows in real-time when a customer has paid, without checking every second.

05 Collections Agent

Monk's AI-powered follow-up system. When an invoice is overdue, this agent writes a personalised email and sends it — then escalates over time if still unpaid. Achieves 24% higher response rates than template emails.

LLM OUTREACH · ESCALATION · RACE CONDITION PREVENTION

THE COLLECTIONS PIPELINE — FROM OVERDUE TO PAID

① Invoice Goes Overdue
Day after due date: scheduler detects unpaid invoice and queues a collections job.

→

② Pre-Flight Check
Re-reads invoice with a database lock. If paid since job was queued — stop. Never email a customer who already paid.

→

③ Build Context
Gathers: customer name, payment history, previous follow-up count, amount owed, days overdue, relationship length.

→

④ AI Writes Email
LLM generates personalised email. Tone adapts: friendly on day 1, formal on day 14, appreciative after partial payment.

→

⑤ Send & Schedule Next
Email sent. Follow-up count incremented. Next check scheduled. After 30 days, Slack alert to finance team.

ESCALATION OVER TIME — WHAT THE AI SENDS DAY BY DAY

+1 Day
Warm, friendly. "Just checking in." Email only.

→

+7 Days
More direct. References the invoice number. Email only.

→

+14 Days
Formal. References contract payment terms. Email + SMS.

→

+30 Days
Agent stops. Slack alert to finance team. Human takes over.

→

Partial Payment
Clock resets to day 0. Warm, appreciative tone. Acknowledges the payment made.

🤖 WHY AI INSTEAD OF TEMPLATES?

Template emails feel robotic and are easy to ignore. Monk's AI writes emails that reference **specific details** — the customer's exact payment history, the amount owed, how long they've been a customer. Monk's own data shows a **24% higher response rate** vs standard dunning emails, which translates directly to faster cash collection for its customers.

⚡ THE RACE CONDITION PROBLEM

At 9am, Monk queues a follow-up email. At 9:02am, the customer pays. At 9:05am, the job fires — without a pre-flight check, it sends a dunning email to someone who just paid. This destroys trust. Monk solves this by re-reading and locking the invoice row immediately before sending. If status has changed to "paid," it silently exits. This is the #1 trust-destroying bug in AR software.

🔄 WHAT IS A STATE MACHINE?

A state machine is a way of modelling something that can only be in one state at a time, with strict rules about how it can change. An invoice moves through states: You cannot go from "paid" back to "outstanding." Monk enforces these rules in the database — no code path can ever accidentally put an invoice in an invalid or contradictory state. This is what makes the system trustworthy for financial data.

06 Reporting Service

Gives CFOs and finance teams a real-time view of their AR health — DSO, aging buckets, and cash collected — without writing a single spreadsheet formula. Dashboard answers are pre-computed so they load instantly.

DSO · AGING BUCKETS · PRE-COMPUTED SNAPSHOTS · ERP SYNC

SNAPSHOT ARCHITECTURE — PRE-COMPUTE ONCE, READ INSTANTLY

Payment & Invoice Events
Every time something changes (payment received, invoice sent), a message is posted to the queue automatically.

→

Snapshot Worker
Listens for events. Recalculates DSO, aging totals, and cash collected. Updates the ar_snapshots table — one row per company.

→

AR Snapshots Table
One pre-calculated row per company (tenant). Contains DSO, aging bucket totals, and cash figures — always ready to read instantly.

→

Dashboard API
When a CFO opens the dashboard, Monk reads one row. Returns in milliseconds. No slow calculations at read time.

📊 DSO — DAYS SALES OUTSTANDING

The average number of days between sending an invoice and receiving payment. **Lower = better.** If DSO is 45, it takes 45 days on average to get paid. Monk tracks a trailing 90-day window — more meaningful than all-time averages. Goal: get DSO below your payment terms (e.g. below 30 for net-30 contracts).

🔄 MULTI-SOURCE RECONCILIATION

A single payment might appear in three places: **Stripe** (webhook), **NetSuite** (batch sync), and **Monk.** All three must agree. Monk stores each payment with a (source, external_id) pair so the same Stripe payment can never be recorded twice. A nightly job cross-checks all three systems for any discrepancies in amount, status, or currency.

🗂️ AGING BUCKETS

Invoices sorted by how overdue they are: **Current** (not yet due), **1–30 days** (recently late), **31–60 days** (concerning), **61–90 days** (urgent), **90+ days** (at risk of write-off). Updated in real time every time a payment arrives or another day passes with an unpaid invoice.

🔗 WHAT IS ERP SYNC?

ERP (Enterprise Resource Planning) is the financial software companies use for accounting — like **NetSuite, QuickBooks, or SAP.** Monk doesn't replace these tools — it syncs payment data into them automatically, so the accounting team always has an up-to-date ledger without entering every payment by hand.

📈 THE RESULT FOR MONK CUSTOMERS

Monk customers report a , and finance teams save — time previously spent on spreadsheets, manual invoice entry, and chasing payments by email. Real-time DSO and aging data lets CFOs make faster, more confident cash-flow decisions without waiting for month-end reports.

07 Glossary of Technical Terms

Every technical word used in Monk's architecture — explained in plain English with a real-world analogy for each one.

PLAIN ENGLISH · NO JARGON · REAL-WORLD ANALOGIES

🔒 TLS Termination

TLS (Transport Layer Security) is the encryption that puts the padlock in your browser and keeps data private in transit. "Termination" means the API Gateway decrypts the incoming encrypted traffic when it arrives. Internal services then communicate in plain text — which is safe because internal traffic never leaves Monk's private network.

🏦 *Analogy: Like a secure bank vault entrance — visitors outside must use a disguise (encryption), but once inside the secure zone, staff operate normally.*

📬 Message Queue / Event Bus

A shared messaging system where services leave notes for each other instead of calling each other directly. When the Payment Service records a payment, it posts "payment.recorded" to the queue. The Reporting Service and Collections Agent are subscribed — they each receive the message and react independently. One service failing cannot crash the others.

📮 *Analogy: Like a shared office bulletin board. The payment team pins a note: "Invoice #456 paid." The reporting team and collections team both check the board and act — without needing to phone each other.*

🔔 Webhooks

A way for external systems (like Stripe) to automatically notify Monk when something happens by calling a specific URL on Monk's server. Monk doesn't keep asking Stripe "has the payment gone through?" — Stripe calls Monk the moment it does. Monk verifies the call is genuinely from Stripe using an HMAC signature before processing it.

🔔 *Analogy: Like a doorbell. Instead of checking every 5 minutes whether your package arrived, the delivery driver rings when it gets there.*

🔑 JWT — JSON Web Token

A digitally-signed piece of data that proves identity without requiring a password. Monk embeds JWTs in payment link URLs — a signed code containing the invoice ID and company ID. Monk verifies the signature to ensure it hasn't been tampered with. If anyone edits the URL, the signature breaks and Monk rejects it. Tokens expire after 30 days.

🎟️ *Analogy: Like a signed wristband at a concert. The wristband tells security exactly who you are and what you're allowed to access — and the signature means it cannot be forged.*

🚦 Rate Limiting

A control that limits how many requests a user or system can make within a given time window. Monk applies this per company — so one customer making excessive API calls can't slow the system down for everyone else. Implemented using Redis, which tracks request counts in real-time sliding windows.

🚗 *Analogy: Like a toll booth — only so many cars can pass per minute to prevent gridlock on the road beyond.*

🔄 Idempotency

A property meaning: doing the same operation multiple times produces the same result as doing it once. In Monk, if the billing scheduler crashes and restarts, the second run sees the billing event is already "invoiced" and does nothing. Critical for payment systems — duplicate charges would be catastrophic and trust-destroying.

🔌 *Analogy: Like a light switch you can flip "on" multiple times — it's still just "on." Not like adding money to a total, where doing it twice would double the amount.*

⚡ Redis Cache

Redis is an extremely fast in-memory data store. Monk uses it to cache frequently-read data — like the AR snapshot and invoice tokens — so the main database isn't hit on every single request. Cached data expires after a set time (TTL — Time To Live) and is refreshed when new data arrives from the event queue.

🧠 *Analogy: Like a sticky note on your desk with the most common answers, so you don't need to go to the filing cabinet every time someone asks a question.*

🗄️ PostgreSQL

Monk's primary database. A relational database that organises data into tables with rows and columns. PostgreSQL is chosen for its ACID guarantees — every transaction is Atomic, Consistent, Isolated, and Durable. This is non-negotiable for a system that handles real money. It also supports row-level locking, which Monk uses to prevent concurrent payment conflicts.

📁 *Analogy: A filing cabinet with very strict rules — every document is filed correctly, nothing is ever lost, and two people cannot edit the same folder simultaneously.*

08 Glossary — continued

More technical terms from Monk's architecture — each explained simply with an analogy.

🌐 API Gateway

The single entry point for all incoming traffic to Monk. Every request — from Finance dashboards, payment portals, Stripe webhooks, and ERPs — passes through here first. It handles authentication (who are you?), rate limiting (how many requests?), and TLS termination (decrypt the secure traffic). Nothing gets to the services behind it without passing these checks.

🏢 *Analogy: Like a building's reception desk — everyone must check in, show ID, and be directed to the right department. No one enters the back office directly.*

🤖 LLM — Large Language Model

The AI technology behind Monk's contract parsing and email writing. LLMs (like GPT-4 or Claude) are trained on vast amounts of text and can understand and generate natural language. Monk uses them to extract billing terms from contracts and write personalised collection emails — but always validates the output against a schema before acting on it. Low-confidence results go to a human review queue.

🧑‍💼 *Analogy: Like a very well-read assistant who has read millions of contracts and emails — they know what billing terms look like, and can write a polite follow-up in any tone you specify.*

🔐 HMAC Signature

HMAC (Hash-based Message Authentication Code) is a way to verify that a message came from a trusted sender and wasn't altered in transit. Stripe signs every webhook with a secret key shared only with Monk. Monk recomputes the signature on its side — if it doesn't match exactly, the webhook is rejected, preventing fake payment notifications from being processed.

📝 *Analogy: Like a wax seal on an envelope. If the seal is broken or doesn't match the sender's stamp, you know the letter was tampered with after it was sent.*

🔒 SELECT FOR UPDATE

A database command that reads a row and simultaneously locks it — preventing any other process from modifying it until the lock is released. Monk uses this in the Collections Agent immediately before sending an email. If another process is simultaneously recording a payment on the same invoice, the pre-flight check waits for that to finish, then reads the updated status. If "paid," it exits silently.

🚪 *Analogy: Like a single-occupancy bathroom — you check if it's empty AND lock the door in one motion. No one else can enter while you're checking the status.*

🔄 ACID Transactions

Atomic (all operations succeed or none do), **C**onsistent (database stays in a valid state), **I**solated (concurrent transactions don't interfere with each other), **D**urable (committed data survives crashes). Monk's three-way payment write is a single ACID transaction — all three writes happen together, or the database rolls everything back as if nothing occurred.

🏦 *Analogy: A bank transfer — either the money leaves your account AND arrives in theirs, or neither happens. The bank never takes money from you without delivering it to the recipient.*

📋 ERP — Enterprise Resource Planning

The core financial software that companies use to manage accounting and operations — examples include **NetSuite, QuickBooks, SAP, and Xero.** Monk doesn't replace these — it syncs payment and invoice data into them automatically so the accounting team always has an up-to-date ledger without entering anything by hand.

🗂️ *Analogy: Like a company's master filing system. Monk is the assistant who files everything correctly and on time without needing to be asked.*

📊 ClickHouse

A specialised analytics database optimised for reading millions of rows very fast. While PostgreSQL is great for transactional writes, ClickHouse excels at queries like "show me DSO trends over the past 12 months across all 10,000 companies." It stores data in columns rather than rows, making large aggregation queries dramatically faster.

📈 *Analogy: Like a spreadsheet purpose-built for pivot tables — terrible for individual row lookups, incredibly fast at summarising massive datasets.*

🏗️ Multi-Tenancy

Monk serves thousands of companies (called "tenants") on the same infrastructure. Each company's data is completely isolated — a Profound employee can never see ASC's invoices or payments. Every database query includes a `tenant_id` filter as a hard requirement, and PostgreSQL Row Level Security provides an additional layer of protection at the database itself.

🏢 *Analogy: A multi-storey office building where each floor is a different company. They share the elevator and utilities, but each floor is locked — other tenants cannot access yours.*